

Solving At-Least Sequence Constraints in Nurse Rostering Problem using SAT

Hieu Truong Xuan, Quan Nguyen Hong, Nghia Dao Xuan,
Tuyen Kieu Van, Khanh To Van

VNU University of Engineering and Technology, Vietnam National University, Hanoi 123105, Vietnam

Abstract—This paper presents an efficient SAT encoding designed to address “at least” sequence constraints. These constraints are a type of global constraint that have been effectively handled in constraint programming (CP), but remain challenging for SAT-based approaches. Instead of encoding each “at least” cardinality constraint individually, our method encodes sets of these constraints by creating reusable encoding blocks and systematically linking adjacent blocks. This block-based strategy leads to more compact encoding and improves the solver performance for problems with sequence constraints. We evaluated our approach through an experiment on the Nurse Rostering problem. The results demonstrate that our SAT encoding consistently surpasses existing methods in terms of solving time and encoding size. Our work highlights the potential of SAT-based techniques for effectively addressing sequence constraints and introduces new possibilities for SAT modeling of sequence constraints.

Index Terms—Sequence Constraint, SAT Solving, CNF Encoding, Nurse Rostering Problem

I. INTRODUCTION

The sequence constraints are a type of constraint that restricts the number of occurrences of certain values within subsequences of length w drawn from a set of n variables. For example, given a set of 28 boolean variables, a sequence constraint may require that in every 10 consecutive variables, at least 4 and at most 7 variables take the value *true*. The sequence constraints appear in various applications, such as scheduling, operational research, and combinatorial problems. In the Crew Rostering problem, the sequence constraints ensure that crews receive adequate rest by limiting the workload within each 7-day period [Siala et al., 2014]. In the Car Sequencing problem, sequence constraints restrict the number of specific options that can be installed within a subsequence of cars, thereby preventing overloads on the assembly line [Artigues et al., 2014]. The sequence constraints also appear in the Employee Scheduling problem, where they help balance the workload between the morning and afternoon shifts of an employee [Nieuwenhuis et al., 2021]. In addition, the sequence constraints are used to model the Anti-bandwidth problem, which has applications in radio frequency assignment, scheduling, obnoxious facility location, and map coloring [Sinnl, 2021] [Hieu et al., 2025]. The sequence constraints are significant in the Nurse Rostering problem (NRP) [Hoda et al., 2010] [Bergman et al., 2014] [Gentzel et al., 2022], as they regulate the frequency and workload that each nurse must manage over consecutive days. For instance,

each nurse may be required to work a minimum of 4 and a maximum of 8 evening shifts within any 14-day period.

Sequence constraints have been extensively explored in the field of Constraint Programming (CP) to develop efficient solutions. This is highlighted in works such as [Hoda et al., 2010], [Bergman et al., 2014], [Gentzel et al., 2020], and [Gentzel et al., 2022]. These studies demonstrate that MDD (Multi-valued Decision Diagram) propagation is regarded as an effective method for addressing sequence constraints in CP. In this paper, we present an alternative approach to CP for solving sequence constraints, specifically a SAT solving method. We focus on the At-Least-K (ALK) sequence constraint, which requires that in any w consecutive elements of a given set of n variables, at least k variables must be *true*. We refer to this constraint as the At-Least Sequence Constraint (ALSC). To address this type of constraint, we propose a novel SAT encoding that leverages the overlapping sub-expressions among successive ALK constraints, allowing a set of ALK constraints to be encoded collectively rather than individually. Specifically, our encoding first partitions the set of ALK constraints into blocks composed of nested sub-expressions. These blocks are then encoded using the Sequential Counter (SC) encoding [Sinz, 2005], effectively capturing their nested structure. As a result, our encoding significantly reduces the number of clauses and variables compared to encoding each ALK constraint independently.

Our research has two key contributions. First, we introduce an efficient SAT encoding for ALSC that significantly reduces the number of variables and clauses compared to existing SAT encodings that encode each ALK constraint individually. Second, we demonstrate the effectiveness of our proposed encoding by applying it to the Nurse Rostering problem, resulting in improved performance compared to existing approaches, including both SAT-based and non-SAT-based methods (i.e., CP and MIP solvers).

The structure of the paper is organized as follows: Section 1 introduces the research problem and highlights the main contributions of the study. Section 2 provides background information on At-Least Sequence Constraints and presents the SAT encoding technique used to solve these constraints. Section 3 describes the Nurse Rostering problem (NRP), its constraints, and the Integer Linear Programming (ILP) model for the NRP. Section 4 presents the experimental evaluation, comparing our proposed SAT encoding with other SAT encodings, both CP and MIP solvers in CPLEX for the NRP.

Finally, Section 5 concludes the paper.

II. SAT ENCODING FOR AT LEAST SEQUENCE CONSTRAINTS

A. At Least Sequence Constraint - ALSC

An At-Least Sequence Constraint (ALSC) is a set of At-Least-K (ALK) cardinality constraints that have the same width w (i.e., the number of variables per constraint). In an ALSC, each pair of consecutive ALK constraints overlaps in exactly $w - 1$ variables. Specifically, the last $w - 1$ variables of the first ALK constraint are the first $w - 1$ variables of the next one. The formal definition of ALSC is provided in Definition 1.

Definition 1: Given a sequence of n boolean variables $\Omega = \{x_1, x_2, \dots, x_n\}$ and a number w such that $1 < w \leq n$, an ALSC of width w is formulated as follows:

$$ALSC(\Omega, w) = \bigwedge_{i=0}^{n-w} \left(\sum_{j=i+1}^{i+w} x_j \geq k \right) \quad 1 \leq k < w$$

Figure 1 illustrates an ALSC composed of seven At-Least-K cardinality constraints, each with a width of 4. The difference between two consecutive constraints lies in the first variable of the preceding constraint and the last variable of the succeeding one, while the remaining variables are the same. This overlap enables the reuse of common sub-expressions to improve computational efficiency. For example, the sub-expression $x_2 + x_3 + x_4$ can be combined with x_1 and x_5 to form two complete expressions $x_1 + x_2 + x_3 + x_4$ and $x_2 + x_3 + x_4 + x_5$.

B. Decomposition of At Least Sequence Constraint

Leveraging the reuse of overlapping sub-expressions between constraints, as discussed in Section II-A, we decompose each ALK constraint into two separate ALK constraints. Proposition 1 presents the formal formulation for this decomposition.

Proposition 1: The constraint $x_1 + x_2 + \dots + x_w \geq k$ holds if and only if for all m s.t. $1 \leq m < w$:

$$\bigwedge_{i=1}^k \left(\left(\sum_{j=1}^m x_j \geq i \right) \vee \left(\sum_{j=m+1}^w x_j \geq k - i + 1 \right) \right)$$

Proposition 1 can be proven by evaluating all possible values of $\sum_{j=1}^m x_j$, which range from 0 to m . For each specific value v such that $\sum_{j=1}^m x_j = v$, the constraint $\sum_{j=1}^m x_j \geq v + 1$ is *false*. As a result, the disjunction ($\vee$) in Proposition 1 necessitates that the constraint $\sum_{j=m+1}^w x_j \geq k - v$ must be *true*. By combining these two components, we obtain the inequality $\sum_{j=1}^m x_j + \sum_{j=m+1}^w x_j \geq v + (k - v) = k$, which satisfies the original constraint.

Given an ALSC with variable set $\Omega = \{x_1, x_2, \dots, x_n\}$ and a width w , we determine the decomposing points for the ALK constraints (corresponding to the value of m in Proposition 1) as the boundaries between disjoint subsets obtained by dividing Ω into $M = \lceil \frac{n}{w} \rceil$ non-overlapping subsets, each

containing w consecutive variables. Note that the last subset may contain fewer than w variables if $n \bmod w \neq 0$. In Figure 1, the original set of variables can be divided into three subsets $\{x_1, x_2, x_3, x_4\}$, $\{x_5, x_6, x_7, x_8\}$, and $\{x_9, x_{10}\}$. ALK constraints that span the boundaries of these subsets are split into two parts. For example, the constraint $x_2 + x_3 + x_4 + x_5$ is divided into $x_2 + x_3 + x_4$ and x_5 ; similarly, $x_3 + x_4 + x_5 + x_6$ is divided into $x_3 + x_4$ and $x_5 + x_6$. Figure 2 illustrates the decomposition of all ALK constraints in Figure 1 based on the subset partitioning described above.

C. ALSC Encoding

After decomposing the ALSC, we group the resulting nested sub-expressions into blocks (see Figure 2) and use *Sequential Counter* (SC) encoding [Sinz, 2005] to encode the largest expression within each block. This approach not only ensures that all expressions are encoded, but also avoids redundant encodings of nested sub-expressions. For example, in the leftmost block of Figure 2, the expression $x_1 + x_2 + x_3 + x_4$ contains the sub-expressions $x_2 + x_3 + x_4$, $x_3 + x_4$, and x_4 . Therefore, applying SC encoding to $x_1 + x_2 + x_3 + x_4$, using the counting direction $x_4 \rightarrow x_3 \rightarrow x_2 \rightarrow x_1$, implicitly encodes all of these three sub-expressions.

For each block i formed by grouping nested sub-expressions, we denote by $x_{i,j}$ as the j^{th} variable considered by the SC encoding within the variable sequence of block i . For instance, the counting direction of the SC encoding for the leftmost block (i.e., the first block) in Figure 2 is $x_4 \rightarrow x_3 \rightarrow x_2 \rightarrow x_1$. Therefore, we have $x_{1,1} \equiv x_4$, $x_{1,2} \equiv x_3$, $x_{1,3} \equiv x_2$, and $x_{1,4} \equiv x_1$.

We also define $R_{i,j,s}$ as a register bit representing whether the sum of the first j variables in block i reaches a threshold s . Specifically, $R_{i,j,s}$ is *true* if $x_{i,1} + \dots + x_{i,j} \geq s$, and *false* otherwise, that is, when $x_{i,1} + \dots + x_{i,j} \leq s - 1$.

It is necessary to mention that our objective is to satisfy the ALK condition. Consequently, when representing the sum $x_{i,1} + \dots + x_{i,j}$, if $k < j$, it is sufficient to use only k register bits $\{R_{i,j,1}, \dots, R_{i,k,k}\}$. For example, to represent $x_{1,1} + x_{1,2} + x_{1,3} + x_{1,4}$ under the At-Least-Two condition, we only need two register bits, which are $R_{1,4,1}$ and $R_{1,4,2}$.

Figure 3 illustrates the assignment of register bits for the first and second blocks (the leftmost and upper central blocks) of the ALSC illustrated in Figure 2. Taking the first block as an example, the sub-expression x_4 has a maximum value of 1, and thus requires only one register bit, denoted $R_{1,1,1}$. Next, the sub-expression $x_3 + x_4$ has a maximum value of 2, and therefore is assigned two register bits, $R_{1,2,1}$ and $R_{1,2,2}$. For $x_2 + x_3 + x_4$, although its maximum value is 3, due to the constraint ≥ 2 , this sub-expression is also assigned only two register bits, namely $R_{1,3,1}$ and $R_{1,3,2}$. Similarly, the sub-expression $\{x_1 + x_2 + x_3 + x_4\}$ is assigned two register bits despite its maximum value being 4.

Given $x_{i,j}$ and $R_{i,j,s}$ as previously defined, the encoding for block i of width w is constructed using the set of formulas (1) – (7), as detailed below.

$$\begin{array}{l}
\boxed{x_1 + x_2 + x_3 + x_4} \geq k \\
\boxed{x_2 + x_3 + x_4 + x_5} \geq k \\
\boxed{x_3 + x_4 + x_5 + x_6} \geq k \\
\boxed{x_4 + x_5 + x_6 + x_7} \geq k \\
\boxed{x_5 + x_6 + x_7 + x_8} \geq k \\
\boxed{x_6 + x_7 + x_8 + x_9} \geq k \\
\boxed{x_7 + x_8 + x_9 + x_{10}} \geq k
\end{array}$$

Fig. 1. ALSC of 10 variables and width 4.

$$\begin{array}{l}
\boxed{x_1 + x_2 + x_3 + x_4} \\
\boxed{x_2 + x_3 + x_4} \\
\boxed{x_3 + x_4} \\
\boxed{x_4}
\end{array}
+
\begin{array}{l}
\boxed{x_5} \\
\boxed{x_5 + x_6} \\
\boxed{x_5 + x_6 + x_7}
\end{array}
+
\begin{array}{l}
\boxed{x_5 + x_6 + x_7 + x_8} \\
\boxed{x_6 + x_7 + x_8} \\
\boxed{x_7 + x_8}
\end{array}
+
\begin{array}{l}
\boxed{x_9} \\
\boxed{x_9 + x_{10}}
\end{array}$$

Fig. 2. Decomposition of ALSC of 10 variables and width 4.

$$\begin{array}{llll}
\boxed{x_1 + x_2 + x_3 + x_4} & \equiv & R_{1,4,1}, R_{1,4,2} & \\
\boxed{x_2 + x_3 + x_4} & \equiv & R_{1,3,1}, R_{1,3,2} & \\
\boxed{x_3 + x_4} & \equiv & R_{1,2,1}, R_{1,2,2} & \\
\boxed{x_4} & \equiv & R_{1,1,1} & \\
\boxed{x_5} & & & \equiv R_{2,1,1} \\
\boxed{x_5 + x_6} & & & \equiv R_{2,2,1}, R_{2,2,2} \\
\boxed{x_5 + x_6 + x_7} & & & \equiv R_{2,3,1}, R_{2,3,2}
\end{array}$$

Fig. 3. Register bits constructing of the first two blocks of ALSC of 10 variables and width 4 under the At-Least-Two constraint.

$$\begin{aligned}
& \bigwedge_{j=1}^{w_i} x_{i,j} \rightarrow R_{i,j,1} \\
& \bigwedge_{j=2}^{w_i} \bigwedge_{s=1}^{\min(j-1,k)} R_{i,j-1,s} \rightarrow R_{i,j,s} \\
& \bigwedge_{j=2}^{w_i} \bigwedge_{s=2}^{\min(j,k)} x_{i,j} \wedge R_{i,j-1,s-1} \rightarrow R_{i,j,s} \\
& \bigwedge_{j=1}^k \neg x_{i,j} \rightarrow \neg R_{i,j,j} \\
& \bigwedge_{j=2}^{w_i} \bigwedge_{s=2}^{\min(j,k)} \neg R_{i,j-1,s-1} \rightarrow \neg R_{i,j,s} \\
& \bigwedge_{j=2}^{w_i} \bigwedge_{s=1}^{\min(j-1,k)} \neg x_{i,j} \wedge \neg R_{i,j-1,s} \rightarrow \neg R_{i,j,s} \\
& R_{i,w_i,k}
\end{aligned}$$

- (1) Formula (1) stipulates that the sum of the first j variables in block i must be at least one whenever $x_{i,j}$ holds *true*. Formula (2) asserts that if the sum of the first $j-1$ variables in block i is at least s , then the sum of the first j variables remains at least s . Formula (3) ensures that if $x_{i,j}$ is *true* and the sum of the first $j-1$ variables is at least $s-1$, then the sum of the first j variables must be at least s . Collectively, these formulas (1), (2), and (3) ensure the correctness of the statement “ $R_{i,j,s}$ holds *true* if and only if $x_{i,1} + \dots + x_{i,j} \geq s$ ”.

- (4) Next, formula (4) imposes an upper bound, ensuring that if $x_{i,j}$ is *false*, then the sum of the first j variables in block i is strictly less than j . Formula (5) ensures that if the cumulative sum of the first $j-1$ variables is strictly less than $s-1$, then the sum of the first j variables remains below s . Furthermore, formula (6) guarantees that if $x_{i,j}$ is *false* and the sum of the first $j-1$ variables is less than s , then the sum of the first j variables is less than s . Together, the formulas (4), (5), and (6) validate the correctness of the statement “ $R_{i,j,s}$ is *false* if and only if $x_{i,1} + \dots + x_{i,j} \leq s-1$ ”.

- (7) Lastly, formula (7) enforces the constraint that $R_{i,w_i,k}$ must always hold *true*. This requirement guarantees that at least k

TABLE I
NRP CONSTRAINTS.

1.	At most 1 work-shift every day.
2.	At most 6 work-shifts every 7 consecutive days.
3.	At least 4 off-days every 14 consecutive days.
4.	At least 4 evening work-shifts every 14 consecutive days.
5.	At most 8 evening work-shifts every 14 consecutive days.
6.	At least 20 work-shifts every 28 consecutive days.
7.	At most 4 night work-shifts every 14 consecutive days.
8.	At least 1 night work-shift every 14 consecutive days.
9.	At least 2 evening/night work-shifts every 7 consecutive days.
10.	At most 4 evening/night work-shifts every 7 consecutive days.
11.	Night work-shifts are not allowed on successive days.

out of the w_i variables within the block are *true*. Moreover, the size of each clause in our encoding, defined as the number of literals per clause, does not exceed three, thereby maintaining a bounded and efficient representation.

After all blocks are encoded, we connect the corresponding sub-expressions together, using Proposition 1, to reformulate the original ALK constraints. For example, the constraint $x_3 + x_4 + x_5 + x_6 \geq 2$ can be reformulated as follows:

$$\begin{aligned}
& x_3 + x_4 + x_5 + x_6 \geq 2 \\
& \quad \text{Prop.1} \\
& \quad \equiv \\
& (x_3 + x_4 \geq 1 \vee x_5 + x_6 \geq 2) \quad \wedge \\
& (x_3 + x_4 \geq 2 \vee x_5 + x_6 \geq 1) \\
& \quad \equiv \\
& (R_{1,2,1} \vee R_{2,2,2}) \wedge (R_{1,2,2} \vee R_{2,2,1})
\end{aligned}$$

III. NURSE ROSTERING PROBLEM

The Nurse Rostering problem (NRP) focuses on creating a feasible work schedule for n nurses over a planning period of d days. Each day consists of three shifts: day shift (D), evening shift (E), and night shift (N). Each nurse is assigned to work one of the three shifts D, E, or N; or has a day off (O) each day. The constraints of the NRP, based on the work of [Bergman et al., 2014], are detailed in Table I and include various sequence constraints. These constraints cover the requirements for the number and frequency of shifts as well as mandatory rest periods to ensure that nurses have adequate recovery time between work periods.

For certain NRP constraints originally formulated as At-Most constraints, we transform them into equivalent At-Least constraints. Specifically, the 2nd constraint, specified as “At most 6 work-days every 7 consecutive days”, is equivalent to requiring “At least 1 off-day every 7 consecutive days”. The 5th constraint, “At most 8 evening work-shifts every 14 consecutive days”, can be restated as “At least 6 evening off-shifts every 14 consecutive days”. Likewise, the 7th constraint, “At most 4 night work-shifts every 14 consecutive days”, corresponds to “At least 10 night off-shifts every 14 consecutive days”, and the 10th constraint, “At most 4 evening/night work-shifts every 7 consecutive days”, can be reformulated as “At least 10 evening/night off-shifts every 7 consecutive days”.

TABLE II
ILP MODEL OF NRP.

1.	$\sum_{s \in \{D,E,N,O\}} x_{i,j,s} = 1$	$;\forall i \in \{1,n\}, j \in \{1,d\}$
2.	$\sum_{j'=j}^{j+6} x_{i,j',O} \geq 1$	$;\forall i \in \{1,n\}, j \in \{1,d-6\}$
3.	$\sum_{j'=j}^{j+13} x_{i,j',O} \geq 4$	$;\forall i \in \{1,n\}, j \in \{1,d-13\}$
4.	$\sum_{j'=j}^{j+13} x_{i,j',E} \geq 4$	$;\forall i \in \{1,n\}, j \in \{1,d-13\}$
5.	$\sum_{j'=j}^{j+13} \neg x_{i,j',E} \geq 6$	$;\forall i \in \{1,n\}, j \in \{1,d-13\}$
6.	$\sum_{j'=j}^{j+27} \neg x_{i,j',O} \geq 20$	$;\forall i \in \{1,n\}, j \in \{1,d-27\}$
7.	$\sum_{j'=j}^{j+13} \neg x_{i,j',N} \geq 10$	$;\forall i \in \{1,n\}, j \in \{1,d-13\}$
8.	$\sum_{j'=j}^{j+13} x_{i,j',N} \geq 1$	$;\forall i \in \{1,n\}, j \in \{1,d-13\}$
9.	$\sum_{j'=j}^{j+6} x_{i,j',E} + x_{i,j',N} \geq 2$	$;\forall i \in \{1,n\}, j \in \{1,d-6\}$
10.	$\sum_{j'=j}^{j+6} \neg x_{i,j',E} + \neg x_{i,j',N} \geq 10$	$;\forall i \in \{1,n\}, j \in \{1,d-6\}$
11.	$\sum_{j'=j}^{j+1} x_{i,j',N} \leq 1$	$;\forall i \in \{1,n\}, j \in \{1,d-1\}$

Given a nurse i , a day j , and a shift s , we denote $x_{i,j,s}$ as a boolean variable that is *true* if and only if nurse i is assigned to shift s on day j ; otherwise, $x_{i,j,s}$ is *false*. If nurse i works the day (D), evening (E), or night (N) shift on day j , then the corresponding variable $x_{i,j,D}$, $x_{i,j,E}$, or $x_{i,j,N}$ is *true*, respectively. A nurse is considered to have an off-day if and only if that nurse is not assigned to any shift on that day. This condition is represented by $x_{i,j,O}$ being *true* if and only if all the three variables $x_{i,j,D}$, $x_{i,j,E}$, and $x_{i,j,N}$ are *false*. Conversely, $x_{i,j,O}$ is *false* if one of the variables $x_{i,j,D}$, $x_{i,j,E}$, or $x_{i,j,N}$ is *true*. Using this definition, the NRP constraints outlined in Table I can be formally represented through an Integer Linear Programming (ILP) model, as shown in Table II. Each constraint in Table I corresponds to an ILP formula with the same index number.

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

To assess the effectiveness of the proposed ALSCE encoding, referred to as ALSCE (At-Least Sequence Constraint Encoding), we developed a framework to solve the NRP. Our framework was evaluated on 28 NRP instances, where the size of each instance is based on the work [Lei et al., 2021]. Each instance has dimensions of $n * w$, where $n \in \{30, 40, 50, 60, 80, 100, 120\}$ represents the number of nurses and $w \in \{12, 16, 20, 24\}$ represents the number of weeks.

In this framework, we implemented ALSCE encoding alongside three widely used SAT-based encodings, including BDD [Abío et al., 2012], Cardinality Network [Asín et al., 2011] and Totalizer [Bailleux and Boufkhad, 2003]. We also included KMTotalizer [Morgado et al., 2014], an encoding for k-cardinality constraints that is built upon the Totalizer. Note that BDD encoding is incorporated into our framework through the PyPBLib library [Ansótegui et al., 2019], while Cardinality Network, Totalizer, and KMTotalizer encodings are available via the PySAT library [Ignatiev et al., 2018]. In our implementation, these five SAT-based encodings are applied to the 3rd, 4th, 5th, 6th, 7th, 9th and 10th constraints,

TABLE III
NRP EXPERIMENTAL RESULTS OF DIFFERENT APPROACHES.

Instance size	ALSCE	BDD	Card	Totalizer	KMTotalizer	CPLEX _{CP}	CPLEX _{MIP}
30n * 12w	2.94	15.27	16.93	6.51	88.39	TO	6.57
30n * 16w	4.24	19.58	23.67	8.94	176.65	TO	7.89
30n * 20w	5.29	24	28.96	11.16	295.92	TO	13.47
30n * 24w	6.57	27.06	35.17	13.39	527.19	TO	12.73
40n * 12w	3.67	22.74	25.78	7.85	137.63	TO	9.49
40n * 16w	5.44	23.92	30.45	11.44	263.63	TO	9.23
40n * 20w	7.41	51.12	40.78	14.92	529.96	TO	14.97
40n * 24w	8.67	54.84	51.18	17.9	TO	TO	26.21
50n * 12w	5.03	25.08	27.06	9.64	198.35	TO	11.13
50n * 16w	6.97	51.36	44.22	14.14	447.24	TO	19.59
50n * 20w	8.78	32.15	51.4	18.55	TO	TO	19.04
50n * 24w	11	97.19	79.32	22.84	TO	TO	22.16
60n * 12w	5.99	31.98	34.2	11.57	282.32	TO	17.33
60n * 16w	8.06	53.46	56.43	16.92	537.6	TO	24.4
60n * 20w	10.83	131.53	59.32	22.13	TO	TO	31.61
60n * 24w	13.27	115.19	88.34	27.22	TO	TO	30.95
80n * 12w	8.71	45.05	59.68	15.36	510.53	TO	22.01
80n * 16w	11.26	122.27	64.22	23.2	TO	TO	18.23
80n * 20w	14.14	90.81	106.19	29.45	TO	TO	27.61
80n * 24w	17.21	171.79	143.6	36.28	TO	TO	41.08
100n * 12w	10.58	57.96	58.97	19.18	TO	TO	20.46
100n * 16w	13.73	246.06	90.77	29.53	TO	TO	40.15
100n * 20w	17.01	327.45	103.52	36.71	TO	TO	49.83
100n * 24w	21.63	206.91	139.74	45.31	TO	TO	45.21
120n * 12w	11.99	115	76.45	27.18	TO	TO	31.1
120n * 16w	16.56	378.67	106.81	33.73	TO	TO	46.57
120n * 20w	20.8	504.04	131.15	44.19	TO	TO	55.53
120n * 24w	26.01	531.47	179.49	54.47	TO	TO	87.18

while the 1st, 2nd, 8th and 11st constraints are encoded using the Pairwise encoding, as they have a simple structure and involve only a small number of variables.

In addition to SAT-based methods, we compare ALSCE with Constraint Programming (CP) and Mixed Integer Programming (MIP) approaches. Both approaches are implemented using the CP and MIP APIs provided by IBM ILOG CPLEX Optimization Studio¹ version 22.1.1.0. Specifically, the ILP model presented in Table II is directly encoded into the CP and MIP solvers via these APIs, which subsequently perform automated searches for feasible solutions. It is necessary to note that IBM ILOG CPLEX Optimization Studio is a powerful framework that integrates constraint propagation with advanced optimized techniques and has been extensively developed and refined over many years.

All 28 NRP instances are executed on a local machine equipped with an AMD Ryzen 5 4600H processor (up to 3GHz, 6 cores, 12 threads) and 16GB of RAM, with a time limit of 600 seconds imposed on each instance. For the SAT-based methods, the Kissat solver [Fleury and Heisinger, 2020] is used as the underlying SAT solver.

B. Experimental Results

Table III summarizes our experimental results on 28 NRP instances using seven different approaches: ALSCE, BDD, Card (Cardinality Network), Totalizer, KMTotalizer, CPLEX_{CP}, and CPLEX_{MIP}. For each NRP instance, Table III reports the solving time required by each method to find a solution. The result of the best-performing method for each instance is highlighted in bold. If a method fails to find a solution within the 600-second time limit, the result is marked as timeout (TO). Figure 4 provides a visual representation of the results reported in Table III.

¹<https://www.ibm.com/products/ilog-cplex-optimization-studio/cplex-optimizer>

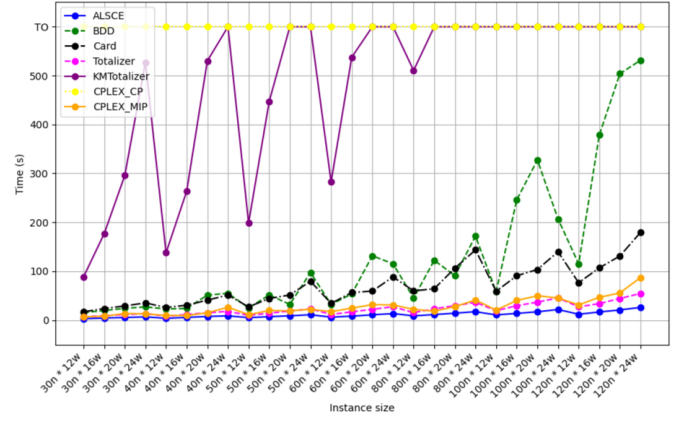


Fig. 4. NRP solving time of different approaches.

The experimental results demonstrate that the ALSCE encoding maintains low and stable solving times as the problem size increases. Specifically, for the smallest instance (30n * 12w), ALSCE finds a solution in just 2.94 seconds. When the number of weeks is doubled (30n * 24w), or the number of nurses is quadrupled (120n * 12w), the solving times increase proportionally to 6.57 and 11.99 seconds, respectively. For the largest instance (120n * 24w), ALSCE solves the problem in 26.01 seconds, roughly twice the time required for the 120n * 12w instance, and about four times that of the 30n * 24w instance.

When comparing ALSCE with the other methods, ALSCE consistently outperforms all alternatives across 28 NRP instances. In addition to ALSCE, Totalizer and CPLEX_{MIP} demonstrate relatively stable performance, with average solving times per instance approximately twice that of ALSCE. Cardinality Network encoding is also a viable option for solving the NRP, although its average solving time is about six times higher than that of ALSCE. It is also worth noting that, despite CPLEX_{CP} and CPLEX_{MIP} employing a parallel search strategy, there is no instance in which they outperform ALSCE. This further reinforces the efficiency of the ALSCE encoding in solving the Nurse Rostering problem.

Figures 5 and 6 compare the number of clauses and variables generated by the SAT-based encodings across all 28 NRP instances. The results show that as the problem size increases, denoted by the number of nurses and weeks, the ALSCE consistently uses fewer clauses and variables than the other encodings. It not only provides an explanation for the superior computational performance of ALSCE but also emphasizes the effective use of memory resources of our proposed encoding. Notably, the growth curves of clause and variable counts for ALSCE exhibit a lower slope compared to the other encodings, demonstrating its scalability to larger problem instances.

V. CONCLUSIONS

The SAT encoding proposed in the paper, ALSCE, is designed to effectively handle At-Least Sequence Constraints by encoding a set of ALK constraints using a block-based

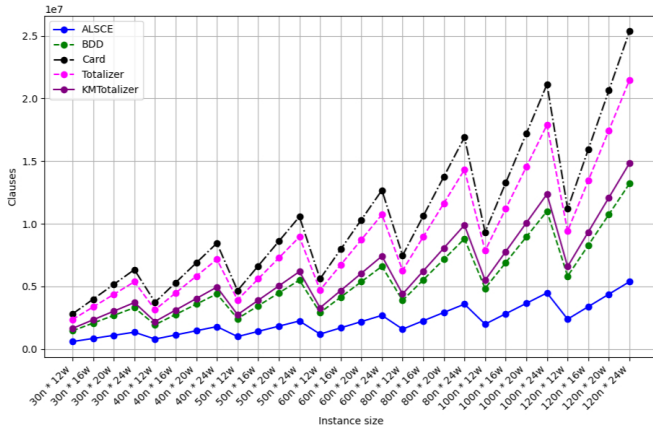


Fig. 5. Clauses required of SAT-based approaches.

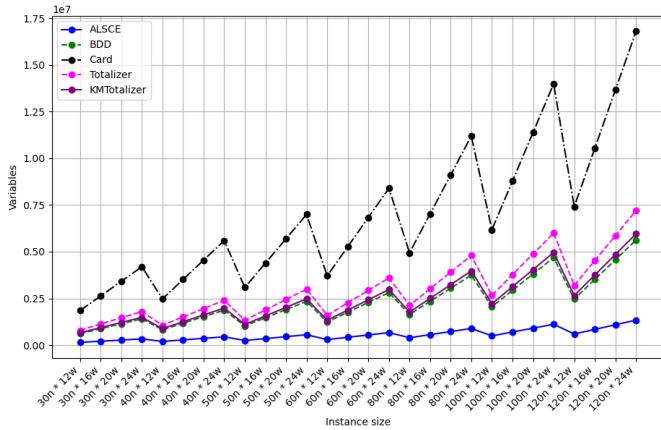


Fig. 6. Total variables used by SAT-based approaches.

strategy. This approach differs from the conventional encoding, which encodes each ALK constraint individually. Experimental results from scheduling problems, such as the NRP, which includes several At-Least Sequence Constraints, demonstrate the effectiveness of our encoding when compared to direct encoding techniques like BDD, Cardinality Network, Totalizer, and KMTotalizer. Furthermore, our encoding demonstrated improved performance over well-established commercial solvers in Constraint Programming and Mixed-Integer Programming, such as CPLEX. This innovative SAT-based representation of sequence constraints presents a promising encoding technique for tackling various scheduling and operations research problems that involve At-Least sequence constraints.

REFERENCES

- [Abío et al., 2012] Abío, I., Nieuwenhuis, R., Oliveras, A., Rodríguez-Carbonell, E., and Mayer-Eichberger, V. (2012). A new look at bdds for pseudo-boolean constraints. *Journal of Artificial Intelligence Research*, 45:443–480.
- [Ansótegui et al., 2019] Ansótegui, C., Pacheco, T., and Pon, J. (2019). Pypbplib (2019). URL: <https://pypi.org/project/pypbplib>.
- [Artigues et al., 2014] Artigues, C., Hebrard, E., Mayer-Eichberger, V., Siala, M., and Walsh, T. (2014). Sat and hybrid models of the car sequencing problem. In *Integration of AI and OR Techniques in Constraint Programming: 11th International Conference, CPAIOR 2014, Cork, Ireland, May 19–23, 2014. Proceedings 11*, pages 268–283. Springer.
- [Asín et al., 2011] Asín, R., Nieuwenhuis, R., Oliveras, A., and Rodríguez-Carbonell, E. (2011). Cardinality networks: a theoretical and empirical study. *Constraints*, 16:195–221.
- [Bailleux and Boufkhad, 2003] Bailleux, O. and Boufkhad, Y. (2003). Efficient cnf encoding of boolean cardinality constraints. In *International conference on principles and practice of constraint programming*, pages 108–122. Springer.
- [Bergman et al., 2014] Bergman, D., Cire, A. A., and van Hoeve, W. (2014). Mdd propagation for sequence constraints. *Journal of Artificial Intelligence Research*, 50:697–722.
- [Fleury and Heisinger, 2020] Fleury, A. and Heisinger, M. (2020). Cadical, kissat, paracooba, plingeling and treengeling entering the sat competition 2020. *Sat Competition*, 2020:50.
- [Gentzel et al., 2020] Gentzel, R., Michel, L., and van Hoeve, W.-J. (2020). Haddock: A language and architecture for decision diagram compilation. In *Principles and Practice of Constraint Programming: 26th International Conference, CP 2020, Louvain-la-Neuve, Belgium, September 7–11, 2020, Proceedings 16*, pages 531–547. Springer.
- [Gentzel et al., 2022] Gentzel, R., Michel, L., and van Hoeve, W.-J. (2022). Heuristics for mdd propagation in haddock. In *28th International Conference on Principles and Practice of Constraint Programming (CP 2022)*.
- [Hieu et al., 2025] Hieu, T. X., Tuyen, K. V., and Khanh, T. V. (2025). Sequential counter encoding for staircase at-most-one constraints. In *Proceedings of the 17th International Conference on Agents and Artificial Intelligence - Volume 2: ICAART*, pages 164–175. SciTePress.
- [Hoda et al., 2010] Hoda, S., Van Hoeve, W.-J., and Hooker, J. N. (2010). A systematic approach to mdd-based constraint programming. In *Principles and Practice of Constraint Programming—CP 2010: 16th International Conference, CP 2010, St. Andrews, Scotland, September 6–10, 2010. Proceedings 16*, pages 266–280. Springer.
- [Ignatiev et al., 2018] Ignatiev, A., Morgado, A., and Marques-Silva, J. (2018). Pysat: A python toolkit for prototyping with sat oracles. In *International Conference on Theory and Applications of Satisfiability Testing*, pages 428–437. Springer.
- [Lei et al., 2021] Lei, Z., Cai, S., and Luo, C. (2021). Extended conjunctive normal form and an efficient algorithm for cardinality constraints. In *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, pages 1141–1147.
- [Morgado et al., 2014] Morgado, A., Ignatiev, A., and Marques-Silva, J. (2014). Mscg: Robust core-guided maxsat solving: System description. *Journal on Satisfiability, Boolean Modelling and Computation*, 9(1):129–134.
- [Nieuwenhuis et al., 2021] Nieuwenhuis, R., Oliveras, A., Rodríguez-Carbonell, E., and Rollon, E. (2021). Employee scheduling with sat-based pseudo-boolean constraint solving. *IEEE access*, 9:142095–142104.
- [Siala et al., 2014] Siala, M., Hebrard, E., and Huguet, M.-J. (2014). An optimal arc consistency algorithm for a particular case of sequence constraint. *Constraints*, 19:30–56.
- [Sinnl, 2021] Sinnl, M. (2021). A note on computational approaches for the antibandwidth problem. *Central European Journal of Operations Research*, 29(3):1057–1077.
- [Sinz, 2005] Sinz, C. (2005). Towards an optimal cnf encoding of boolean cardinality constraints. In *International conference on principles and practice of constraint programming*, pages 827–831. Springer.